

PRD — Plataforma Inteligente para Análise e Comparação de Apólices D&O

- 1. Visão Geral do Produto
- 2. Usuários-Alvo
- 3. Requisitos Funcionais
- 4. Requisitos Não-Funcionais
- 5. Histórias de Usuário
- 6. Arquitetura do Sistema
- 7. Design da Interface (Streamlit — 4 páginas)
- 8. Critérios de Aceite
- 9. Fora do Escopo (Out of Scope)
- 10. Dependências Técnicas
- 11. Cronograma de Entrega
- 12. Riscos e Mitigações
- 13. Decisões Técnicas Justificadas

PRD — Plataforma Inteligente para Análise e Comparação de Apólices D&O

InsurMinds · Projeto Final · I2A2 — Instituto de Inteligência Artificial Aplicada

Campo	Valor
Produto	InsurMinds Apólice Analyzer
Versão do PRD	1.0
Autor	Equipe InsurMinds
Data	01/09/2026
Deadline	06/10/2026 às 23h59
Status	Draft → Aprovado

1. Visão Geral do Produto

1.1 Declaração do Problema

Apólices de seguro **D&O (Directors and Officers)** são documentos extensos (tipicamente 30–80 páginas), redigidos em linguagem jurídica densa e altamente técnica. Comparar manualmente duas ou mais apólices para identificar diferenças em coberturas, exclusões, franquias e limites de responsabilidade demanda **várias horas de trabalho especializado** por parte de corretores, advogados ou executivos.

1.2 Declaração da Solução

O **InsurMinds Apólice Analyzer** é uma plataforma inteligente baseada em IA Generativa que automatiza o processo de extração, estruturação e comparação de apólices D&O. A partir do upload de documentos em PDF, o sistema extrai automaticamente as informações relevantes, armazena-as de forma estruturada e apresenta um relatório comparativo claro e acessível ao usuário.

1.3 Objetivo do Produto

Demonstrar, em um MVP funcional, a capacidade de integrar OCR, LLMs, agentes inteligentes e interfaces de consulta para resolver um problema real do mercado segurador, como trabalho de conclusão do curso InsurMinds I2A2.

1.4 Métricas de Sucesso

Métrica	Critério mínimo de aceite
Extração de texto	Extrair texto de PDFs digitais com 100% de cobertura
Identificação de cláusulas	Identificar ≥ 5 categorias de informação relevantes por apólice
Comparação	Comparar ≥ 2 apólices e apresentar diferenças
Performance	Processar 1 apólice em ≤ 60 segundos
Usabilidade	Usuário consegue completar o fluxo sem instruções externas
Demonstração	Sistema pode ser demonstrado ao vivo em até 5 minutos

2. Usuários-Alvo

2.1 Personas

Persona 1 — Corretor de Seguros Especializado

- **Contexto:** Trabalha com seguros corporativos, recebe múltiplas propostas de diferentes seguradoras
- **Dor:** Precisa comparar apólices D&O manualmente, processo leva horas
- **Objetivo:** Ver rapidamente quais são as coberturas e exclusões de cada proposta
- **Expectativa:** Interface simples, resultado claro e objetivo

Persona 2 — Executivo / Diretor (segurado)

- **Contexto:** CFO ou Diretor Jurídico que precisa entender o que sua apólice cobre

- **Dor:** Não entende linguagem jurídica das apólices
- **Objetivo:** Saber em linguagem simples o que está coberto e o que não está
- **Expectativa:** Linguagem acessível, resumo executivo claro

■ Persona 3 — Avaliador do Curso (professor/banca)

- **Contexto:** Avalia o projeto como trabalho de conclusão do InsurMinds
 - **Dor:** Quer ver aplicação real dos conceitos aprendidos (agentes, LLMs, OCR, dados)
 - **Objetivo:** Verificar arquitetura consistente, código organizado e solução funcional
 - **Expectativa:** Demonstração clara do funcionamento, documentação técnica completa
-

3. Requisitos Funcionais

3.1 RF01 — Upload de Documentos

- O sistema **DEVE** aceitar upload de arquivos PDF
- O sistema **DEVE** aceitar múltiplos arquivos simultaneamente (mínimo 2, máximo 5 por sessão)
- O sistema **DEVE** validar o formato do arquivo antes de processar
- O sistema **DEVE** informar o status do processamento em tempo real (barra de progresso)
- O sistema **DEVE** limitar o tamanho máximo de arquivo em 50MB por documento

3.2 RF02 — Extração de Conteúdo

- O sistema **DEVE** extrair texto de PDFs com conteúdo digital (texto nativo) usando `pdfplumber`
- O sistema **DEVE** detectar automaticamente quando um PDF é escaneado (imagem) e acionar fallback

- O sistema **DEVE** usar Gemini Vision como fallback para PDFs escaneados ou com baixa qualidade de texto
- O sistema **DEVE** preservar a estrutura lógica do documento (seções, cláusulas)

3.3 RF03 — Identificação e Estruturação de Cláusulas

O sistema **DEVE** identificar e extrair as seguintes categorias de informação:

Campo	Descrição
<code>segurado</code>	Nome da pessoa jurídica segurada
<code>seguradora</code>	Nome da seguradora emissora
<code>numero_apolice</code>	Número único da apólice
<code>vigencia_inicio</code>	Data de início da vigência
<code>vigencia_fim</code>	Data de término da vigência
<code>premio_total</code>	Valor do prêmio total
<code>limite_responsabilidade</code>	Limite máximo de cobertura
<code>franquia</code>	Valor ou percentual de franquia
<code>coberturas</code>	Lista de coberturas incluídas
<code>exclusoes</code>	Lista de exclusões expressas
<code>clausulas_especiais</code>	Cláusulas particulares ou aditivos
<code>retroatividade</code>	Data de retroatividade da cobertura
<code>territorio</code>	Abrangência territorial da cobertura
<code>legislacao_aplicavel</code>	Legislação e foro de eleição

- O sistema **DEVE** estruturar os dados em formato JSON padronizado (schema Pydantic)
- O sistema **DEVE** indicar confiança/incerteza quando uma informação não for encontrada

3.4 RF04 — Armazenamento

- O sistema **DEVE** persistir as apólices processadas em banco de dados SQLite local

- O sistema **DEVE** evitar reprocessamento de apólices já armazenadas (identificação por hash MD5)
- O sistema **DEVE** permitir listar as apólices já processadas
- O sistema **DEVE** permitir excluir apólices armazenadas

3.5 RF05 — Comparação entre Apólices

- O sistema **DEVE** comparar pelo menos 2 apólices selecionadas pelo usuário
- O sistema **DEVE** apresentar uma **tabela comparativa** campo a campo
- O sistema **DEVE** destacar visualmente as diferenças encontradas
- O sistema **DEVE** calcular e apresentar um **score de similaridade** entre as apólices
- O sistema **DEVE** identificar coberturas presentes em uma apólice mas ausentes em outra

3.6 RF06 — Relatório Comparativo

- O sistema **DEVE** gerar um relatório narrativo em linguagem natural (via Gemini)
- O relatório **DEVE** incluir: resumo executivo, principais diferenças, recomendação objetiva
- O sistema **DEVE** permitir exportar o relatório em texto formatado
- O relatório **DEVE** ser gerado em Português do Brasil

3.7 RF07 — Interface de Usuário

- O sistema **DEVE** ter uma interface web acessível via navegador
- A interface **DEVE** ter as seguintes páginas/seções:
 1. **Upload** — Envio de documentos PDF
 2. **Biblioteca** — Apólices já processadas
 3. **Comparação** — Seleção e visualização comparativa
 4. **Relatório** — Análise narrativa do Gemini
- A interface **DEVE** indicar o progresso de processamento

- A interface **DEVE** exibir mensagens de erro claras e acionáveis
-

4. Requisitos Não-Funcionais

4.1 Performance

- Extração de texto nativo (pdfplumber): ≤ 5 segundos por documento
- Análise via Gemini (identificação + estruturação): ≤ 45 segundos por documento
- Geração do relatório comparativo: ≤ 30 segundos
- **Total end-to-end para 2 apólices:** ≤ 2 minutos

4.2 Usabilidade

- Usuário completa o fluxo completo (upload → comparação → relatório) em ≤ 5 cliques
- Interface em Português do Brasil
- Feedback visual em todos os estados (carregando, sucesso, erro)

4.3 Confiabilidade

- Tratamento de erros em todos os agentes (falha não deve crashar a aplicação)
- Fallback automático de extração (pdfplumber → Gemini Vision)
- Logging de todas as etapas do pipeline

4.4 Segurança (escopo mínimo)

- Chaves de API armazenadas apenas em variáveis de ambiente (`.env` , nunca em código)
- `.gitignore` configurado para excluir `.env` e dados de apólices sensíveis
- Arquivos de apólices não compartilhados no repositório público

4.5 Manutenibilidade

- Código organizado em módulos por responsabilidade
- Cada agente em arquivo separado
- Docstrings em funções públicas
- README.md com instruções completas de instalação e uso

5. Histórias de Usuário

Épico 1: Ingestão de Documentos

ID	Como...	Quero...	Para...
US-01	usuário	fazer upload de um PDF de apólice D&O	iniciar o processamento automático
US-02	usuário	ver o progresso do processamento	saber que o sistema está trabalhando
US-03	usuário	receber alerta quando o arquivo for inválido	corrigir antes de perder tempo
US-04	usuário	ver a apólice processada na biblioteca	reutilizá-la em comparações futuras

Épico 2: Extração e Estruturação

ID	Como...	Quero...	Para...
US-05	usuário	ver os dados extraídos da apólice	verificar se a extração foi correta
US-06	usuário	saber quando um campo não foi encontrado	entender as limitações da extração
US-07	sistema	reprocessar PDFs escaneados via Gemini Vision	garantir cobertura completa de documentos

Épico 3: Comparação

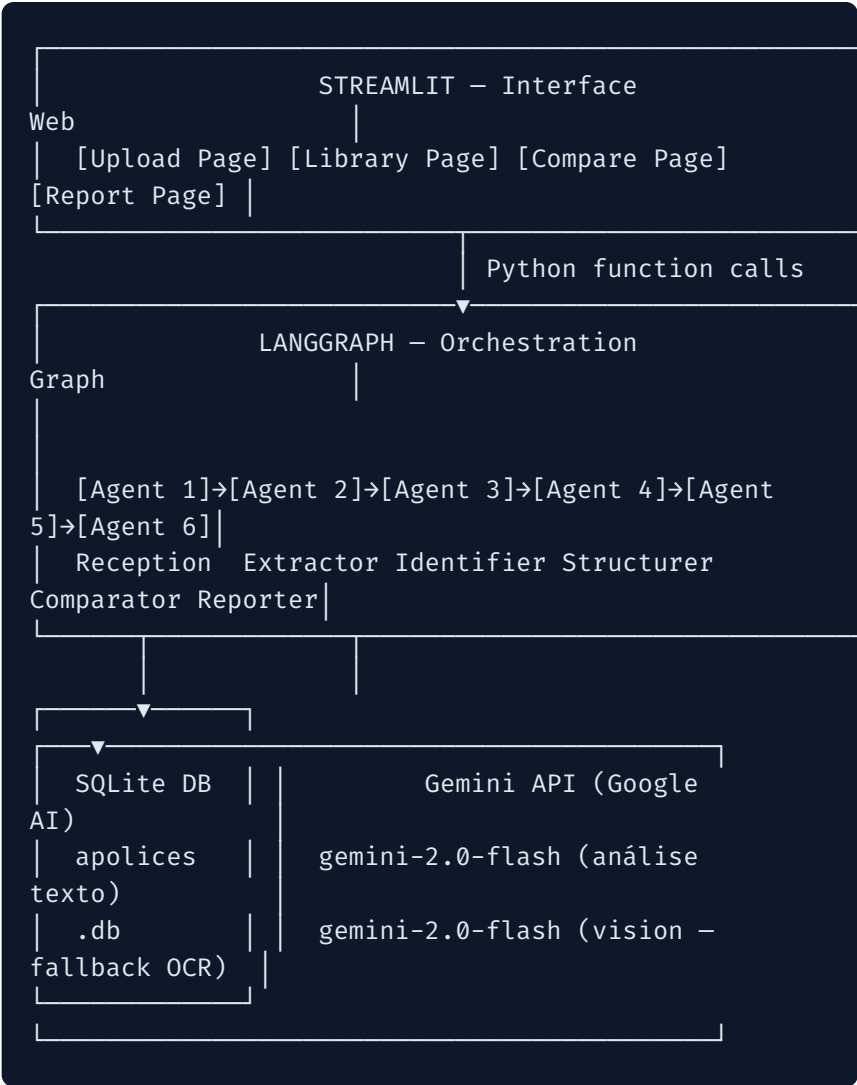
ID	Como...	Quero...	Para...
US-08	usuário	selecionar 2 apólices da biblioteca	iniciar uma comparação
US-09	usuário	ver uma tabela com as diferenças campo a campo	identificar rapidamente o que muda
US-10	usuário	ver as diferenças destacadas em cores	focar no que é relevante
US-11	usuário	ver quais coberturas uma apólice tem e a outra não	tomar decisão de contratação

Épico 4: Relatório

ID	Como...	Quero...	Para...
US-12	usuário	ler um resumo em linguagem simples das diferenças	entender sem precisar de especialista
US-13	usuário	copiar ou exportar o relatório	compartilhar com gestor ou cliente

6. Arquitetura do Sistema

6.1 Visão Macro



6.2 Descrição dos Agentes LangGraph

Agente 1 — Reception Agent

- **Responsabilidade:** Validar e preparar o documento recebido
- **Input:** Arquivo PDF (bytes)
- **Output:** Metadados do arquivo (nome, tamanho, hash MD5, número de páginas)
- **Validações:** formato PDF, tamanho $\leq 50\text{MB}$, arquivo não corrompido
- **Estado LangGraph:** `DocumentState.file_info`

Agente 2 — Extractor Agent

- **Responsabilidade:** Extrair texto bruto do documento
- **Input:** Caminho do arquivo PDF
- **Output:** Texto bruto por página + flag `is_scanned`
- **Estratégia:**
 1. Tenta `pdfplumber` (texto nativo)
 2. Se texto < 100 chars/página → aciona Gemini Vision (fallback)
- **Estado LangGraph:** `DocumentState.raw_text`, `DocumentState.extraction_method`

Agente 3 — Identifier Agent

- **Responsabilidade:** Identificar seções e cláusulas relevantes no texto bruto
- **Input:** Texto bruto da apólice
- **Output:** Texto segmentado por categoria (coberturas, exclusões, etc.)
- **Tecnologia:** Gemini com prompt estruturado de segmentação
- **Estado LangGraph:** `DocumentState.identified_sections`

Agente 4 — Structurer Agent

- **Responsabilidade:** Converter seções identificadas em JSON estruturado
- **Input:** Seções identificadas pelo Agente 3
- **Output:** `ApoliceDAO` (objeto Pydantic validado)
- **Tecnologia:** Gemini com `response_schema` (structured output)
- **Estado LangGraph:** `DocumentState.structured_data`

Agente 5 — Comparator Agent

- **Responsabilidade:** Comparar duas ou mais apólices estruturadas
- **Input:** Lista de objetos `ApoliceDAO`

- **Output:** `ComparisonResult` com diffs campo a campo e score de similaridade
- **Estratégia:** Comparação direta de campos + Gemini para análise semântica de listas
- **Estado LangGraph:** `ComparisonState.diff_result`

Agente 6 — Reporter Agent

- **Responsabilidade:** Gerar relatório narrativo em linguagem natural
- **Input:** `ComparisonResult`
- **Output:** Relatório em markdown (PT-BR)
- **Tecnologia:** Gemini com prompt de geração de relatório executivo
- **Estado LangGraph:** `ComparisonState.report`

6.3 Schema de Dados — ApoliceDAO

```

from pydantic import BaseModel
from typing import Optional

class ApoliceDAO(BaseModel):
    # Identificação
    id: Optional[str] = None          # Hash MD5 do
arquivo
    nome_arquivo: str
    data_processamento: str

    # Dados da apólice
    numero_apolice: Optional[str] = None
    segurado: Optional[str] = None
    seguradora: Optional[str] = None
    vigencia_inicio: Optional[str] = None
    vigencia_fim: Optional[str] = None

    # Valores financeiros
    premio_total: Optional[str] = None
    limite_responsabilidade: Optional[str] = None
    franquias: Optional[str] = None

    # Coberturas e exclusões
    coberturas: list[str] = []
    exclusoes: list[str] = []
    clausulas_especiais: list[str] = []

    # Escopo
    retroatividade: Optional[str] = None
    territorio: Optional[str] = None
    legislacao_aplicavel: Optional[str] = None

    # Metadados de extração
    metodo_extracao: str
    # "pdfplumber" | "gemini_vision"
    confianca_extracao: float          # 0.0 a 1.0
    campos_nao_encontrados: list[str] = []

```

6.4 Schema de Dados — ComparisonResult

```
class FieldDiff(BaseModel):
    campo: str
    valor_apolice_a: Optional[str]
    valor_apolice_b: Optional[str]
    ha_diferenca: bool
    tipo_diferenca: str # "valor", "ausente_a",
    "ausente_b", "igual"

class ComparisonResult(BaseModel):
    apolice_a_id: str
    apolice_b_id: str
    data_comparacao: str
    score_similaridade: float # 0.0 a 1.0
    diffs: list[FieldDiff]
    coberturas_exclusivas_a: list[str]
    coberturas_exclusivas_b: list[str]
    coberturas_comuns: list[str]
    excludoes_exclusivas_a: list[str]
    excludoes_exclusivas_b: list[str]
```

7. Design da Interface (Streamlit — 4 páginas)

Página 1 — Upload (Enviar Apólice)

The screenshot displays the 'InsurMinds Apólice Analyzer' application. At the top, the title bar reads 'Análise Inteligente de Apólices D&O'. Below the title bar, there is a large rectangular area with a dashed border containing the text: 'Arraste seus PDFs aqui ou clique para selecionar' and 'Suporte: PDF • Máx: 50MB por arquivo'. Below this area, a section titled 'Processar Apólice(s)' contains a progress bar and the text 'Processando: 80% - Agente 4'. Below the progress bar, there is a list of tasks: 'Extração concluída', 'Cláusulas identificadas', and 'Estruturando dados ...'. The interface is dark-themed with white text and a dashed border around the main content area.

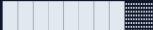
InsurMinds Apólice Analyzer

Análise Inteligente de Apólices D&O

Arraste seus PDFs aqui
ou clique para selecionar

Suporte: PDF • Máx: 50MB por arquivo

► Processar Apólice(s)

Processando:  80% — Agente 4

- Extração concluída
- Cláusulas identificadas
- Estruturando dados ...

Página 2 — Biblioteca (Apólices)

Apólices Processadas (3)			
Arquivo	Seguradora	Data	Ações
allianz	Allianz	01/09	 
chubb	Chubb	01/09	 
aig	AIG	01/09	 

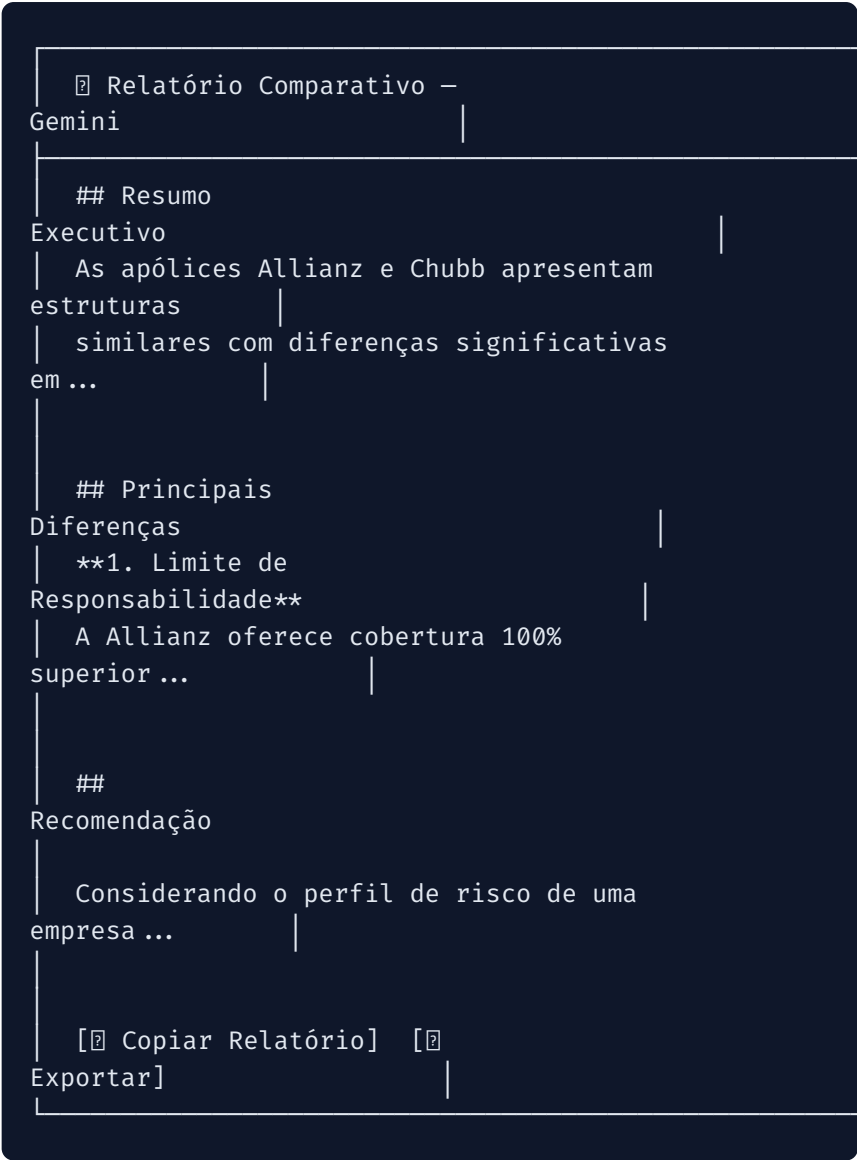
☒ Allianz
 ☒ Chubb
 ☐ AIG

 ► Comparar Seleccionadas

Página 3 — Comparação (⚖️ Comparar)

Comparação: Allianz x Chubb			
Similaridade:		78%	
Campo Chubb	Allianz		
Limite Resp.	R\$ 10M	R\$ 5M	▲
DIFERENTE			
Franquia	R\$ 50K	R\$ 100K	▲
DIFERENTE			
Retroatividade	3 anos	5 anos	▲
DIFERENTE			
Território	Brasil	Brasil	🇧🇷
IGUAL			
Prêmio	R\$ 120K	R\$ 85K	▲
DIFERENTE			
Coberturas exclusivas Allianz: [Investigação Regulatória]			
Coberturas exclusivas Chubb: [Side A DIC]			

Página 4 — Relatório (Relatório)



8. Critérios de Aceite

8.1 Obrigatórios (entrega completa)

- ☐ Solução executa sem erros após `pip install -r requirements.txt && streamlit run app.py`
- ☐ Upload e processamento de PDF funciona end-to-end
- ☐ Extração automática retorna dados estruturados
- ☐

Comparação entre ≥ 2 apólices funciona e apresenta diferenças

- ☐ Relatório narrativo é gerado pelo Gemini
- ☐ Repositório GitHub público e acessível
- ☐ README.md contém: descrição, instalação, execução, tecnologias, integrantes, licença MIT
- ☐ Pasta `Projeto_Final_Artefatos/` existe no repositório com os artefatos

8.2 Desejáveis (nota máxima)

- ☐ Fallback Gemini Vision funciona para PDFs escaneados
 - ☐ Score de similaridade calculado e exibido
 - ☐ Apólices persistidas e reutilizáveis entre sessões
 - ☐ Tratamento de erros com mensagens claras ao usuário
 - ☐ Logging completo do pipeline
 - ☐ Testes automatizados básicos passam
-

9. Fora do Escopo (Out of Scope)

Item	Justificativa
Integração com sistemas de seguradoras	Não é objetivo do curso
Autenticação de usuários	Fora do escopo do MVP
Multi-tenancy	Aplicação single-user
Deploy em produção (cloud)	MVP local é suficiente
Suporte a outros tipos de apólice (auto, vida, etc.)	Escopo restrito a D&O
Processar mais de 5 apólices simultaneamente	Limitação de MVP
Assinaturas digitais ou validação jurídica	Fora do escopo técnico
Interface mobile / responsiva	Desktop first

10. Dependências Técnicas

10.1 Dependências Python (requirements.txt)

```
streamlit ≥ 1.39.0
langgraph ≥ 0.2.0
langchain-google-genai ≥ 2.0.0
pdfplumber ≥ 0.11.0
pydantic ≥ 2.0.0
python-dotenv ≥ 1.0.0
Pillow ≥ 10.0.0
pdf2image ≥ 1.17.0
sqlalchemy ≥ 2.0.0
pytest ≥ 8.0.0
```

10.2 Variáveis de Ambiente (.env)

```
GOOGLE_API_KEY=sua_chave_aqui
GEMINI_MODEL=gemini-2.0-flash
GEMINI_VISION_MODEL=gemini-2.0-flash
MAX_FILE_SIZE_MB=50
DB_PATH=./data/apolices.db
LOG_LEVEL=INFO
```

10.3 Pré-requisito do sistema

```
sudo apt-get install -y poppler-utils # Necessário
para pdf2image (Ubuntu/Debian)
```

11. Cronograma de Entrega

Entregável	Prazo interno
PRD aprovado	Semana 1 (atual)
Estrutura do projeto + requirements	Semana 1
Pipeline de extração funcional (Agentes 1-2)	Semana 1
Agentes 3-4 (identificação + estruturação)	Semana 2
Agentes 5-6 + comparação	Semana 3
Interface Streamlit completa	Semana 3
Testes + tratamento de erros	Semana 4
Relatório Técnico	Semana 5
Pitch Deck + Vídeo	Semana 5
Entrega final no GitHub	06/10/2026

12. Riscos e Mitigações

Risco	Prob.	Im- pacto	Mitigação
PDF com layout complexo (tabelas, colunas)	Média	Alto	Fallback Gemini Vision
Rate limit da API Gemini	Baixa	Médio	Cache de resultados no SQLite
Campo não encontrado na apólice	Alta	Médio	Validação mínima bloqueia comparação se campos obrigatórios (<code>limite_responsabilidade</code> , <code>coberturas</code> , <code>exclusoes</code> , <code>vigencia_inicio</code> , <code>vigencia_fim</code>) estiverem vazios + highlight visual ⚠ para campos secundários ausentes (custo zero — lógica Python pura)
Gemini retorna JSON mal formatado	Média	Médio	<code>response_schema</code> + validação Pydantic + retry
PDF corrompido	Baixa	Médio	Validação no Agente 1 antes de processar

13. Decisões Técnicas Justificadas

Decisão	Alternativas	Justificativa
Streamlit	FastAPI+JS, Gradio	Curva baixa, experiência prévia, perfeito para MVP demonstrável
LangGraph	CrewAI, LangChain linear	Controle explícito do grafo, alinhado com sugestão do professor
Gemini	OpenAI, Claude	Já em uso no workspace, tier free generoso, vision integrado
pdfplumber	PyMuPDF, pypdf	Melhor extração de tabelas e layout, licença MIT
SQLite	PostgreSQL, MongoDB	Zero infraestrutura, perfeito para MVP local, sem custo operacional
Pydantic	dataclasses, dict	Validação automática, integração nativa com LangGraph, schema claro